PROTOTYPE A — ENVIRONMENTAL CLEARING

Roblox Minimum Functional Fantasy Specification | v1.0

Status: PROPOSED FOR OWNER REVIEW; NOT IMPLEMENTED / NOT PLAYTESTED
Date: 2026-11-06
Prototype code: REST-A
Primary fantasy: "I can turn a messy space into a visibly clean space through my own actions."
Reference products (inspiration only): *Clean the Moon!*, *Clean all the Leaves*, *Mow The Lawn!*
First-pass effort assumption: 2–3 focused developer-days (6 hours/day), plus participant recruitment and real-world playtesting.
Shared authority: `05_Shared_Research_and_Playtest_Protocol_v1.0.md`. No relationship to *Thaw a Creature*.

1. Decision this prototype must answer

Can removing simple, repeated debris patches be intrinsically satisfying enough that a new Roblox player completes one cleanup and voluntarily starts another, without progression or currency?

- **Hypothesis A-H1:** Spatially localized removal plus strong before/after readability motivates completion without an external reward.
- **Hypothesis A-H2:** A new debris layout reuses the system cheaply enough for many content variations.
- **Failure criterion:** Most participants treat cleaning as repetitive labor, require an external reward, cannot tell which area remains dirty, or do not want another round after finishing.
- **Do not infer:** Success proves only the *clearing* interaction; it does not prove a monetizable simulator or long-term retention.

2. Locked minimum scope (P0)

Area	P0 requirement	Explicitly excluded
World	One flat 48 x 48 stud yard, a start pad, one bounded cleanup plot (~24 x 24 studs)	Open world, biomes, quests, decoration pass

Area	P0 requirement	Explicitly excluded
Mess	60 dirt/debris patches initially arranged on an 8 x 8 logical grid (four empty), 3 simple visual variants	Physics-heavy trash, hundreds of loose parts, rarity
Core input	Point at a patch; hold/tap Clean; receive immediate localized feedback	Tool upgrades, multiple equipment types
Patch state	Dirty (2 valid hits) → Half-clean (1 hit remaining) → Clean (0)	Dirt textures that require GPU pixel painting
Progress	0–100% based on *total hits completed*; clear area visually	Coins, XP, inventory, shops, rebirth
Finish	One short clean-area celebration; “Try another round” or “Finish”	Daily rewards, leaderboards, forced replay
Replay	Reset same yard with one of 3 deterministic seeded layouts	Additional authored maps
Data	Server-authoritative round and patch counters in memory	DataStore, cross-session progress

First-pass tuning values (not final design rules): 60 patches x 2 accepted hits = 120 successful actions; server accepts at most 7 cleaning actions per second; reach <=12 studs; target first completion 45–90 seconds. Theoretical fastest action time is less than actual completion time because movement and aiming matter. Tune based on recorded sessions rather than forcing players to wait.

3. Player-facing interaction contract

Player entry: Spawn facing the dirty yard. A small label says **“Clean the yard: aim at the mess and hold Clean.”** A visible `0% CLEAN` counter appears; tool and nearest patch must be obvious.

Action: The player targets an unclean patch and activates the Clean action. A valid hit changes the exact patch (e.g., dark → pale → disappears). Soft particles and an unobtrusive sound confirm acceptance. Invalid targeting causes no progress. The player can move while cleaning.

Completion: On the last valid patch hit, the whole yard appears cleared and the HUD displays `100% CLEAN` with an optional brief before/after camera view. Show equally visible buttons **“Try another round”** and **“Finish.”** No performance pressure or timed event.

Replay: A new round seeds the same patch pool into another prevalidated arrangement. Reset visuals and counters atomically; do not spawn a second yard or accumulate effects.

Input by device

- **Desktop:** mouse look / reticle target; hold left mouse button or keyboard action to clean.
- **Mobile:** keep the crosshair centered, provide a dedicated large Clean touch button, allow normal camera movement with the other thumb. A tap mode is an acceptable accessibility alternative; measure which option participants use.

- **Gamepad:** optional stretch. The core interaction must not require fine pixel accuracy.
- **Feedback:** use patch disappearance and a numeric counter, not color alone.

4. Level and content handoff

Asset	Quantity / constraint	Acceptance
Yard base	1 simple rectangular surface	Dirty area visible from spawn; no navigation bottleneck
Debris prefabs	3 small variations reused 60 times	Clearly dirty against ground, no need for unique textures
Cleanable marker	60 server-tracked logical cells / models	Every cell stable ID and targetable collision/query proxy
Tool	1 simple broom / scrubber	Legible in first-person and third-person; simple feedback
HUD	1 percentage + prompt + completion panel	Readable on 16:9 desktop and portrait/narrow mobile viewport
Feedback	1 brief cleaning burst + 1 sound + finish effect	No heavy particle spam or camera shake
Replay layouts	3 saved seed patterns	No inaccessible patches / off-ground pieces

Spatial rule: The player should see an already-clean section building up next to a dirty section. This contrast is more important than impressive prop detail. Avoid piling patches into occluded corners. Use dark/bright value differences that work in outdoor lighting.

5. State, data and network contract

Round state machine

```
READY -> ACTIVE -> COMPLETE -> REPLAY_OFFERED -> RESETTING -> ACTIVE
```

Each `RoundState` holds `RoundId`, `LayoutSeed`, `PatchRemainingHits[PatchId]`, `TotalAcceptedHits`, `StartedAt`, and a config version. `TotalHitsTarget = 120` for the first test. Progress is derived as `TotalAcceptedHits / TotalHitsTarget`, **not** a number supplied by the client.

Server validation

A `CleanRequest(RoundId, PatchId)` requests one attempted clean. The server must verify, in this order:

1. Sender is live, alive, in the ACTIVE round, and is not in a reset transition.
2. The `RoundId` matches; `PatchId` belongs to the active yard and still has remaining hits.
3. Sender is within 12 studs of the patch and has a reasonably unobstructed interaction line to its query proxy.
4. Sender's action interval is at least `1 / 7 sec`; duplicate frames and spam do not skip states.

5. Decrement **one** hit, advance authoritative progress **once**, and publish the patch's new state.
6. When all patches reach zero, complete the round exactly once and block further progress.

The client cannot send its own percentage, force a finish, choose a new seed, or create valid patch IDs. Cosmetic local previews may appear immediately but must reconcile with server accepted/rejected messages.

Performance: Prefer a single raycast per active tool sample and small anchored/queryable patch proxies. Avoid 60 independent per-frame scripts, physics, or particle emitters. Cap active visual bursts and reuse effects. `CollectionService` tags (e.g., `RestorationPatch`) may identify spawned patch models. Keep the visual patches anchored and noncolliding so walking remains smooth.

Reset: Restore every patch's hit count and visuals, clear pending client feedback, rotate the `RoundId`, and reset the timer before accepting actions. Character reset mid-round returns the tester to READY (or a fresh round; choose one and keep it consistent).

6. Suggested small module organization

```
ReplicatedStorage/RestA/Config.lua
ReplicatedStorage/RestA/Remotes/CleanRequest
ReplicatedStorage/RestA/Remotes/PatchStateChanged
ReplicatedStorage/RestA/Remotes/RoundStateChanged
ServerScriptService/RestA/RoundService.lua
ServerScriptService/RestA/PatchService.lua
ServerScriptService/RestA/RequestGuards.lua
StarterPlayer/StarterPlayerScripts/RestAClient.lua
Workspace/RestA_Arena/Yard/PatchNodes/*
```

Names are illustrative. Use one bootstrap Script and a few modules; do not create generic services for hypothetical future products.

7. Production sequence and hard cuts

Step	Expected focus	Exit condition
A-01	Blockout yard, patch IDs and 1 cleaning action	One targeted patch changes state correctly
A-02	Server action validation, all 60 patches and progress	Spam / distant targets cannot skip progress
A-03	First-play HUD, particles/sound, completion/replay	Full round works end to end, resets correctly
A-04	Mobile control pass and 3 layout seeds	Can finish on phone without hidden instructions
A-05	Bug fixes and test logging	Acceptance suite passes, no added features

Cut order if late: three layout seeds -> decorative VFX -> extra tool animation.
Do not cut: visible local removal, complete transformation, replay choice, server authority, mobile action affordance.

8. Acceptance tests (A-T01 to A-T12)

ID	Test	PASS condition
A-T01	First entry	Spawn shows full dirty yard, obvious Clean action, `0%`
A-T02	First valid hit	Only targeted patch moves from dirty to half-clean; server hit tally +1
A-T03	Second hit	Same patch becomes clean; cannot be further cleaned
A-T04	Wrong/empty target	No progress and no phantom cleanup
A-T05	Too far / occluded	Forged request rejected; no progress
A-T06	Rapid-fire requests	Rate guard prevents accelerated progress; no double count
A-T07	Completion	Exactly 120 accepted valid hits resolve 100% and one completion event
A-T08	Replay	New RoundId restores visuals and counters; stale actions rejected
A-T09	Character reset / exit	No stuck held action or leaked event connections
A-T10	Touchscreen	A new player can move, aim and clean on a small touch layout
A-T11	Performance	60 patches and effects remain responsive in real/mobile emulation testing
A-T12	Second variation	Another valid seeded layout works without modifying code

9. Measurement and decision gate

Record common metrics in `05_Shared_Research_and_Playtest_Protocol_v1.0.md`: time to first valid hit, completion, replay choice, satisfaction, actual engineering hours, and authoring minutes for a second layout. Also record **time spent hunting the final 10%** and which patches were missed. The last dirty patch should not turn an otherwise relaxing game into tedious pixel hunting.

Proceed to deeper product research only if: most testers understand the clean action quickly, the ground visibly improves, a meaningful subset voluntarily replays, and authoring new layouts is genuinely inexpensive. **Stop or change approach if:** users simply sweep mindlessly without enjoyment, search time dominates, or the motion feels too imprecise.

10. Risks, fallbacks and AI guardrails

Risk	First fallback
Player cannot find last patches	Use subtle highlight on remaining patches at >=90% progress; record how often needed
Client requests spoof progress	Server owns cell list, distance checks, action rate, and completion
Cleaning feels like clicking checkboxes	Increase patch visual response before adding extrinsic rewards
Touch reticle too difficult	Increase query proxies; test tap-to-clean mode as an explicitly logged variant
60 patches create microstutter	Pool effects, reduce part count, remove needless collisions

Instructions to Claude / Codex implementing this document:

1. Do not modify *Thaw a Creature* files or shared Roblox places. Create a separate test place and inspect current content before editing.
2. Implement **P0 only**. Do not assume cash, upgrades, auto-cleaners, rarity, or inventories are approved.

3. Treat all target counts, ranges, durations and acceptance numbers as tunable experiment values; centralize them in Config.
4. Keep the server authoritative over patch state and round completion; do not trust client hit totals.
5. Do not report `PASS` from reading code alone: launch Studio and verify desktop/mobile behavior. If Studio is not accessible, state **NOT TESTED**.
6. Stop at the agreed implementation boundary. Submit actual observed time and defects before recommending expansions.

11. References

Technical API documentation:

- https://create.roblox.com/docs/workspace/raycasting
- https://create.roblox.com/docs/reference/engine/classes/CollectionService
- https://create.roblox.com/docs/reference/engine/classes/ContextActionService

Competitive inspirations are context only; none of their original creation times or complete production requirements are established by this prototype spec.